Incognito

Anonymiser des corpus qualitatifs en local
dans le navigateur — détection, revue, graphe de personnes

CamemBERT · GLiNER · Albert API · UDPipe
— pipelines client-side pour les SHS.

Xiaoou Wang · Ingénieur en Humanités Numériques
MSHS Sud-Est · Université Côte d'Azur
Incognito Web 1.0.0 · AGPLv3 · ~20 minutes

Plan (20 minutes)

	Bloc	~min
1	Problème & proposition	3
2	Architecture web (client-side)	2
3	Pipeline **Basic** (CamemBERT)	3
4	Pipeline **Advanced** (GLiNER)	3
5	Pipeline **Generative** (Albert API)	2.5
6	Graphe de personnes	3
7	Limites & perspectives	1.5

Fil : chaque mode = un pipeline différent ; la revue humaine est commune.

1 · Problème & proposition

Anonymiser sans trahir le terrain

- Entretiens / notes : prénoms, lieux, employeurs, santé, diplômes…
- Risque de **réidentification** par combinaison de détails
- Tension : protéger **et** garder un texte analysable
- Les LLM cloud « marchent » — en **exportant** le corpus

> Incognito vise un autre contrat : **traitement local** + **contrôle humain** + **traçabilité**.

Proposition produit

Détecter → revoir → remplacer → exporter

[PER_1]
[LOC_1]
[ORG_1]
[EMA_1]
[NAT_1]

· 3 lettres de catégorie + index stable

Principe	Implémentation web
Local-first	ONNX / WASM dans le navigateur (sauf Albert)
Humain dans la boucle	Soft-exclude, spans manuels, Related to
Traçabilité	Audit Markdown, CSV graphe
Multi-protocoles	3 moteurs, 1 couche de revue

Assistant d'anonymisation — pas une garantie d'anonymat total.

2 · Architecture web

Stack client-side

```
Interface dans le navigateur  →  détection en arrière-plan (sans figer l'écran)  →  revue, graphe de personnes, exports
```

Couche	Tech · Framework	Rôle
UI	React (Vite), FR/EN	Revue 3+1, batch, progress
Basic NER	Transformers.js + ONNX Runtime WASM	Token-classification CamemBERT / BERT
Advanced	`gliner` + ONNX `q4f16` (~472 Mo)	Labels ouverts, chunking
Generative	`fetch` → Worker CORS → Albert `/v1`	JSON graphe compact
Graphe	UDPipe WASM French-GSD (~23 Mo)	Deps + merge proximité
Hosting	GitHub Pages (statique) + Cloud EOSC	

Où vont les données ?

Basic & Advanced

Texte **reste** dans le navigateur
Download one-shot des poids HF
puis **cache** navigateur

Generative

Texte + clé → **Albert**
via Worker EOSC (European Open Science Cloud)

`Pages github.io → Worker /v1/* → albert.api.etalab.gouv.fr`

3 · Pipeline Basic

CamemBERT / Transformers.js

Basic — flux de traitement

On part du texte collé ou importé
→ le modèle repère les noms, lieux, organisations, dates… *sans bloquer l'interface*
→ chaque mention détectée est regroupée en entités lisibles
→ **toutes les occurrences** de la même valeur sont surlignées (pas seulement la première)
→ les e-mails, URL, dates et téléphones peuvent aussi être trouvés par règles simples
→ le tout est rangé par **catégories** (personne, lieu…)
→ on remplace par des codes du type `[PER_1]` et on montre un aperçu
→ en parallèle, on construit le **graphe de personnes** (possibilité de révision humaine)

Modèles typiques : CamemBERT + dates (défaut), CamemBERT base, BERT anglais

4 · Pipeline Advanced

GLiNER multi-label

Advanced — flux de traitement

On choisit les types à chercher (personne, maladie, diplôme, nationalité…)
→ le modèle GLiNER se télécharge une fois, puis reste en cache dans le navigateur
→ pour e-mail, URL et téléphone, on préfère des **règles explicites** au modèle
→ les résultats alimentent les catégories à l'écran *et* les attributs du graphe de personnes
→ ensuite : même revue que le mode Basic (exclure, rattacher, exporter)

Advanced — pourquoi GLiNER ici

CamemBERT (Basic)	GLiNER (Advanced)
Types NER fixés du modèle	Étiquettes **texte libre** / protocole
Bon défaut FR entretiens	Santé, diplôme, fonction, nationalité…
Plus stable sur PER/LOC/ORG	Plus flexible, parfois plus bruyant

- Seuil bas (0.2) → rappel élevé → **la revue filtre**
- Même couche graphe : attributs plus riches (adresse, profession, école…)

Fichiers : `glinerRuntime.js` · `useGlinerWorker.js` · `ruleDetection.js` · `GlinerWorkflowSection.jsx`

5 · Pipeline Génératif

Albert API

Generative — flux de traitement

On repère d'abord localement des candidats (noms, contacts…)
→ le texte part vers **Albert** (avec la clé de l'utilisateur)
→ Albert renvoie un **petit plan** : qui est qui, alias, attributs, liens → le navigateur rattache ce plan aux mots du texte
→ on construit l'aperçu anonymisé localement
→ l'utilisateur peut exclure, rattacher (« Related to »), ou se concentrer sur une personne
→ chaque correction recalcule l'aperçu (pour éviter des restes du type `[PER_1] Martin`)

Generative — contraintes réseau & tokens

- **CORS** : Albert ne répond pas correctement aux preflights depuis `github.io` → Worker proxy allowlist
- **Proxy** : forward `Authorization` + body ; **aucune clé stockée** côté Cloud
- **max_tokens** / JSON compact : réduire `finish_reason=length`

Fichiers : `albertClient.js` · `albertCandidates.js` · `albertEdit.js` · `albert-proxy/`

6 · Graphe & revue

Couche commune aux 3 modes

Construction du graphe de personnes

On part des entités détectées (Basic / Advanced) ou du plan Albert
→ analyse linguistique du français (dépendances) dans le navigateur
→ on relie chaque mention à sa place dans le texte
→ on regroupe ce qui appartient à la **même personne** :
  · liens grammaticaux dans la phrase
  · proximité dans le paragraphe
  · (Advanced) types riches : adresse, métier, école…
→ l'utilisateur peut corriger avec **Related to** (ces choix sont conservés)
→ exclure une valeur dans les catégories ou dans le graphe : **même effet** sur l'aperçu

Sémantique des interactions

Action	UI surlignée	Texte anonymisé
Pastille personne (chip)	Focus / soulignement	**Aucun effet**
Soft-exclude (× / menu)	Grisé	**Retiré** du remplacement
Related to	Réattache attr / alias	Rebuild + remplacements

Layout 3+1 : Catégories · Texte · Graphe — preview anonymisé en **plein largeur dessous**

Exports : CSV long (1 ligne / surface) · CSV by person · audit · Label Studio

8 · Limites & perspectives

Limites techniques

- Quasi-identifiants / contexte narratif hors NER
- Graphe **incomplet**
- GLiNER / CamemBERT : frontières, faux + / faux –
- WASM : perf machine-dépendante

> Toujours **relire** avant diffusion ou archivage.

Perspectives

- Mobiliser des modèles génératifs pour détecter les quasi-identifiants complexes, notamment ceux qui ne peuvent être repérés par une simple reconnaissance d'entités nommées.
- Coupler l'analyse qualitative avec des méthodes quantitatives d'anonymisation, en exploitant le graphe personnes–informations pour mesurer le risque de réidentification (k-anonymat, unicité des profils, combinaison d'attributs, etc.).
- Évaluer la robustesse de l'anonymisation par des attaques de réidentification, en croisant avec des sources publiques (INSEE, Wikidata, annuaires et autres bases ouvertes) afin d'estimer le risque réel de réidentification.

Takeaways

1. **Trois pipelines** distincts (CamemBERT · GLiNER · Albert+client)
2. **Une couche de revue** (graphe, soft-exclude, Related to, CSV)
3. **Local-first** dans le navigateur + Generative

Liens

- App : xiaoouwang.github.io/Incognito
- Code : github.com/xiaoouwang/Incognito
- Journal : `docs/WEB_INTERFACE_EVOLUTION.md`

Merci

Questions ?

Xiaoou Wang · MSHS Sud-Est · Université Côte d'Azur
xiaoouwang.github.io/Incognito
AGPLv3 · Incognito Web 1.0.0